CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

Chandubhai S Patel Institute of Technology

DEPSTAR

ADVANCED WEB DEVELOPMENT FRAMEWORKS (ITUE301)

Practical 11: Containerization with Docker and Docker Compose

CO/PO	CO5 / PO3, PO5
Objective	To containerize the full-stack application using Docker and orchestrate services using Docker Compose.
Prerequisites	• Practicals 1–10 completed working full-stack Task Management application • Docker Desktop (or Docker Engine) installed and verified on the lab machine
Theory Concepts (Self-Study Reference)	
Coursera Pointer	See Week 11 IBM: Introduction to Containers with Docker, Kubernetes & OpenShift (focus only on the Docker fundamentals and Docker Compose sections; Kubernetes/OpenShift content is out of scope for this course).
Architecture / Diagram	

Diagram	`docker-compose.yml`
	`├── frontend (React, built + served, port 5173)`
	`├── backend (Node/Express, port 5000)`
	`└── mongodb (official mongo image, port 27017)`
	
	`Network: app-network (bridge)`
	`frontend → backend → mongodb`
	`All 3 services start together via: docker-compose up`

Problem Definition	
Problem Statement	• Write a Dockerfile for the React frontend of the Task Management application. • Write a Dockerfile for the Node/Express backend. • Create a docker-compose.yml file that orchestrates all 3 services: frontend, backend, and MongoDB. • Configure correct port mappings and an internal Docker network so the services can communicate. • The entire application must start with a single docker-compose up command, with no manual steps in between.
Key Questions / Analysis	• Why does the backend container need to reference the MongoDB container by its service name (e.g., mongodb) rather than localhost? • What is the difference between EXPOSE in a Dockerfile and the ports mapping in docker-compose.yml? • Why is it considered bad practice to copy node_modules into a Docker image instead of running npm install inside the container?
Supplementary Problems	• Add a .dockerignore file to exclude node_modules and .env from the build context. • Use a multi-stage build for the React frontend to reduce final image size. • Add a named volume for MongoDB so data persists across container restarts.
Key Skills Addressed	• Writing Dockerfiles for both a frontend and backend Node-based application • Orchestrating multi-container applications using Docker Compose • Configuring inter-container networking and port mapping

	• Running and verifying a complete application stack with a single command
Learning Outcome	Students will be able to containerize a full-stack application using Docker and orchestrate multiple services together using Docker Compose, running the entire stack with one command.

Lab Session Step-by-Step

Steps

1. Verify Docker is installed and running:

```
docker --version
docker compose version
```

2. Create a Dockerfile in the backend folder:

```
FROM node:18
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 5000
CMD ["node", "server.js"]
```

3. Create a Dockerfile in the frontend folder:

```
FROM node:18 AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build
EXPOSE 5173
CMD ["npm", "run", "preview", "--", "--host"]
```

4. Create docker-compose.yml at the project root defining all 3 services with build context, ports, and depends_on.
5. Update the backend MongoDB connection string to use the service name (e.g., mongodb://mongodb:27017/taskdb) instead of localhost.
6. Run the entire stack:

```
docker-compose up --build
```

7. Confirm all 3 containers are running and the app is accessible end-to-end:

```
docker ps
```

Tools / Technology / Resources Required	• Docker Desktop / Docker Engine • Docker Compose • Node.js (for local reference, not required at runtime once containerized) • Terminal / Command line
Total Hours	• Lab Session: 2 hours • Implementation: 1.5 hours • Testing / Modification: 0.5 hours • Total Engagement: 2 hours

Faculty Teaching Guide

Guidance Notes	• This is an awareness-level practical, not a production Docker deep dive the goal is for students to see and run a working multi-container setup, not master Docker internals. • Pre-pull the node:18 and mongo images on lab machines before the session if internet bandwidth is limited, to avoid losing lab time to image downloads. • Demonstrate the localhost vs service-name distinction live this is the single most common point of confusion when containerizing a previously local app. • Reserve the last 15 minutes specifically for the single-command verification (docker-compose up) as a class checkpoint.
Common Mistakes	• Using localhost or 127.0.0.1 in the backend's MongoDB connection string instead of the Compose service name. • Forgetting to expose/map the correct ports, making the app inaccessible from the host browser.

	• Not excluding node_modules via .dockerignore, resulting in extremely slow and bloated image builds. • Running docker-compose up without --build after changing a Dockerfile, so old cached layers are used.

Troubleshooting Guide

Symptom	Likely Cause	Fix
Backend cannot connect to MongoDB	Connection string uses localhost instead of the Compose service name	Update MONGO_URI to use the service name defined in docker-compose.yml, e.g. mongodb://mongodb:27017/taskdb
Port already in use error	A previous container or local process is already bound to that port	Run docker ps to find and stop the conflicting container, or change the host-side port mapping
Build is extremely slow	node_modules being copied into the build context	Add a .dockerignore file with node_modules listed
Changes to code not reflected after rebuild	Old image layers cached	Run docker-compose up --build to force a fresh image build

Student Assignment / Post Laboratory Work

Tasks	• Review the Theory Concepts section and the assigned Coursera module before next week. • Add a .dockerignore file excluding node_modules and .env. • Add a named volume for MongoDB so task data persists across container restarts.

GitHub Deliverables

Expected Repository Contents	• Same repository as previous practicals, updated with new commits • Dockerfiles for both frontend and backend, plus a working docker-compose.yml • README updated with instructions to run the full stack via docker-compose up • At least one commit specifically for containerization

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
Conceptual understanding (2 Marks)	Explains what and why, not just what was done
Implementation (4 Marks)	Code written, runs, and is original
Correctness (2 Marks)	Output matches expected result
Reflection (1 marks)	Errors documented and explained
Lab file evidence (1 marks)	Completeness and presentation
Total: 10 marks	**Passing Threshold: 5 / 10 (50%)**